



**UNIVERSITATEA
TEHNICĂ
DIN CLUJ-NAPOCA**

DOOM rulat pe ESP32-S3 cu input prin bluetooth

Proiectare cu MicroProcesoare

Asistent de laborator: Andrei Diaconescu

Student: Stoica Mihai-Nicusor

Grupa: 30237

FACULTATEA DE AUTOMATICA
SI CALCULATOARE

2 Decembrie 2025

Cuprins

1	Abstract	2
2	Componente Hardware	3
2.1	Unitate Principala (Game Engine)	3
2.2	Unitate Secundara (I/O Controller)	3
2.3	Tabel Conexiuni (Pinout)	3
3	Schema Sistemului	4
4	How to Use (Mod de Utilizare)	5
4.1	Configurare Initiala	5
4.2	Conectare Controller	5
4.3	Controale in Joc	5
4.4	HUD Secundar	5
5	How to Do (Detalii Implementare)	6
5.1	Configurare ESP-IDF (Pentru ESP32-S3)	6
5.2	Partitionare Flash (Storage)	6
5.3	Modificari Software: Game Engine	6
5.4	Modificari Software: I/O Controller	6
A	Cod sursa si scheme	8

1 Abstract

Acest proiect prezinta o implementare avansata a jocului clasic DOOM pe un sistem embedded distribuit, bazat pe arhitectura microcontrollere-lor ESP32. Inovatia principala consta in utilizarea a doua unitati de procesare distincte care comunica asincron:

1. Un **ESP32-S3** dedicat exclusiv motorului grafic si logicii jocului (Game Engine).
2. Un **ESP32 Standard** care actioneaza ca I/O Controller, gestionand conexiunea Bluetooth cu un DualShock 4 si un ecran secundar OLED pentru telemetrie.

Aceasta separare a responsabilitatilor permite rularea jocului la un framerate stabil, eliberand procesorul principal de sarcini costisitoare precum stiva Bluetooth si actualizarea display-ului secundar. Proiectul demonstreaza utilizarea eficienta a memoriei PSRAM externe, partitionarea customizata a memoriei flash pentru stocarea fisierelor WAD si comunicarea inter-chip prin UART.

2 Componente Hardware

Sistemul este alcatuit din urmatoarele module interconectate:

2.1 Unitate Principala (Game Engine)

- **Platforma:** ESP32-S3 (WROOM-1 / S3-Zero).
- **Resurse:** 240MHz Dual Core, 16MB Flash, 8MB PSRAM (Octal SPI).
- **Display Principal:** TFT LCD 2.4" sau 1.54", driver ST7789, interfata SPI.
- **Audio:** I2S DAC (ex. MAX98357A sau intern) pentru sunet stereo.

2.2 Unitate Secundara (I/O Controller)

- **Platforma:** ESP32 WROOM-32 (Standard).
- **Resurse:** Bluetooth Classic/BLE suportat hardware.
- **Display Secundar (HUD):** OLED 0.96" sau 1.3", driver SH1106G/SSD1306, interfata I2C.
- **Intrare:** Gamepad Sony DualShock 4 (conectat via Bluepad32).

2.3 Tabel Conexiuni (Pinout)

1. Conexiuni ESP32-S3 (Doom Engine):

Periferic	Pin (GPIO)	Funcție
TFT Display	13	MOSI (Data)
TFT Display	18	SCLK (Clock)
TFT Display	15	CS (Chip Select)
TFT Display	2	DC (Data/Command)
TFT Display	4	RST (Reset)
UART TX	17	Transmisie date (Health/Ammo)

2. Conexiuni ESP32 Standard (Controller):

Periferic	Pin (GPIO)	Funcție
OLED Display	27	SDA
OLED Display	26	SCL
UART TX	17	Transmisie Input catre S3
UART RX	16	Receptie Telemetrie de la S3
Simulare Butoane	18, 19, 21, 22...	Iesiri digitale mapate intern

3 Schema Sistemului

Diagrama de mai jos ilustreaza arhitectura dual-core distribuita. ESP32-ul secundar (dreapta) preia datele de la controller via Bluetooth si le trimite serial catre ESP32-S3 (stanga), care randeaza jocul si trimite inapoi starea jucatorului pentru a fi afisata pe OLED.

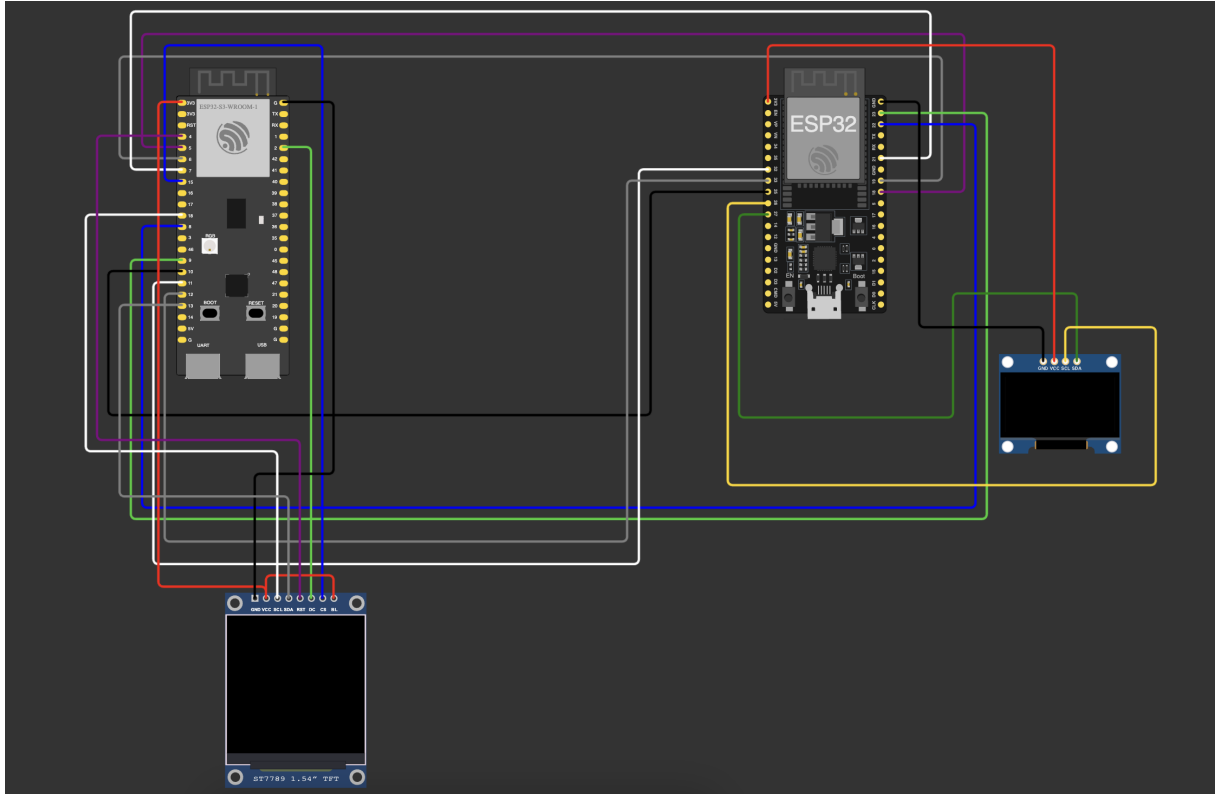


Figura 1: Schema electrica detaliata si fluxul de date

4 How to Use (Mod de Utilizare)

4.1 Configurare Initiala

1. Alimentati ambele placi ESP32 (S3 si Standard) la o sursa USB stabila.
2. Pe ecranul principal (TFT) va aparea secventa de boot DOOM.
3. Pe ecranul mic (OLED) va aparea mesajul: "Bluepad32 Init... Connect Gamepad...".

4.2 Conectare Controller

1. Porniti controllerul.
2. Activati modul de imperechere tinand apasat simultan butoanele **PS** si **SHARE** timp de 3 secunde (led-ul va clipi rapid).
3. ESP32-ul secundar se va conecta automat.
4. Pe OLED va aparea confirmarea: "Controller Connected".

4.3 Controale in Joc

Sistemul mapeaza automat butoanele controller-ului la actiunile din DOOM:

- **D-Pad / Analog Stanga:** Miscare (W, A, S, D).
- **X:** Tragere (Fire / CTRL).
- **O:** Actiune/Deschidere usi (Use / Space).
- **Options:** Meniu / Enter.
- **Share:** Back / Escape.

4.4 HUD Secundar

Urmariti ecranul OLED in timpul jocului pentru informatii critice:

- **Health:** Procentajul vietii ramase.
- **Ammo:** Munitia pentru arma curenta.
- **Butoane:** Indica ce se apasa pe controller.

5 How to Do (Detalii Implementare)

Aceasta sectiune descrie pasii tehnici pentru reproducerea proiectului, configurarea mediului si modificarile aduse codului sursa.

5.1 Configurare ESP-IDF (Pentru ESP32-S3)

Jocul ruleaza pe framework-ul ESP-IDF. Fisierul `sdkconfig` este crucial pentru performanta:

- **Target:** `CONFIG_IDF_TARGET="esp32s3"`
- **Frecventa CPU:** Setata la maxim: `CONFIG_ESP_DEFAULT_CPU_FREQ_MHZ_240=y`
- **Memorie PSRAM:** Activata obligatoriu pentru a incarca texturile si sunetele:

```
CONFIG_SPIRAM=y
CONFIG_SPIRAM_MODE_OCT=y (sau Quad in functie de placa)
CONFIG_SPIRAM_SPEED_80M=y
```

- **Flash Speed:** `CONFIG_ESPTOOLPY_FLASHMODE_QIO=y` pentru citire rapida.

5.2 Partitionare Flash (Storage)

Fisierul `Doom1.WAD` (4MB) trebuie stocat in memoria flash. S-a folosit o tabela de partitii custom (`partitions.csv`) pentru a aloca spatiu suficient:

#	Name,	Type,	SubType,	Offset,	Size,	Flags
nvs,		data,	nvs,	,	0x6000,	
phy_init,		data,	phy,	,	0x1000,	
factory,		app,	factory,	,	1M,	
rofs,		data,	undefined,,		14M,	

Partitia `rofs` este montata ca un sistem de fisiere read-only, permitand motorului de joc sa acceseze resursele direct prin mapare in memorie.

5.3 Modificari Software: Game Engine

Codul sursa din `examples/doom/main.c` a fost modificat fata de repo-ul original *mcugdx*:

1. **Input Redirection:** Functia `DG_GetKey` nu mai citește pinii GPIO locali. Ea interpreteaza pachete primite pe UART de la ESP32-ul secundar.
2. **Telemetry Output:** In functia `DG_DrawFrame`, variabilele interne ale jocului sunt serializate si trimise pe TX: `[language=C] player_t*plr = players[consoleplayer]; printf("Health :`

5.4 Modificari Software: I/O Controller

Codul Arduino utilizeaza biblioteca `Bluepad32` si `Adafruit_SH110X`.

- **Loop Principal:** Verifica conexiunea Bluetooth. Daca exista date, apeleaza `processGamepad(ct1)`.
- **Mapping:** Valorile citite de la `ct1->dpad()` si `ct1->buttons()` sunt traduse in semnale digitale pe pinii conectati la RX-ul ESP32-S3 (sau trimise ca pachete seriale, in functie de versiunea implementata).
- **Display Update:** Parseaza string-urile primite pe Serial ("Health: ...") si le afiseaza pe OLED.

Referințe

- [1] KOTerra, *utcn-DOOM-ESP32S3-dual-screen Repository*, GitHub, 2024.
<https://github.com/KOTerra/utcn-DOOM-ESP32S3-dual-screen>
- [2] Mario Zechner, *mcugdx - Cross platform game development for microcontrollers*, GitHub.
<https://github.com/badlogic/mcugdx>
- [3] Ricardo Quesada, *Bluepad32 - Bluetooth gamepad support for ESP32*, GitHub.

A Cod sursa si scheme